Understanding a Spiking Neuron

André Furlan
Rua Cristóvão Colombo 2265, Jd Nazareth, 15054-000, São José do Rio Preto - SP, Brazil.
Instituto de Biociências, Letras e Ciências Exatas, Unesp - Univ Estadual Paulista (São Paulo State University)
São José do Rio Preto, Brazil
Email: ensismoebius@gmail.com

***Abstract*—**
The growing demand for increased processing power with deeper neural networks results in escalating energy consumption. Spiking Neural Networks (SNNs) emerge as efficient alternative to conventional neural networks, especially when implemented on neuromorphic hardware, which mimics the operating principles of biological neurons to reduce energy usage. Although frameworks like SNNTorch provide ready-to-use tools, understanding the theoretical foundations and practical implementation of SNNs remains essential, particularly for deployment in resource-constrained environments or on specialized architectures where such frameworks may not be feasible. This tutorial presents the fundamental concepts of spiking neurons and guides the reader through their implementation.

***Index Terms*—Spiking Neural Networks, SNN, LIF**

I. Introduction

This guide assumes that you already have some knowledge about neural networks and its inner workings.

Standard Neural Networks (NNs), as defined here — **multilayered, fully connected, with or without recurrent or convolutional layers using sigmoidal and or RElu like activations functions** — require that all neurons be activated during both forward and backward passes. This means that every unit in the network must perform computations at every time step, leading to significant power consumption [1]. In an era where responsible use of energy resources is essential, a critical question arises: How can one design NN models that are energy-efficient enough to be deployed on portable or resource-constrained devices?

Spiking Neural Networks (SNNs) offer a compelling answer to this problem. As the third generation of neural networks, SNNs encode information using discrete spike events and update neuron states only when necessary. This event-driven paradigm mimics biological neuronal dynamics and allows for sparse computations, which are particularly advantageous when implemented on neuromorphic hardware. Such systems can achieve high energy efficiency by leveraging the inherent sparsity of spike-based communication.

Although frameworks like SNNTorch provide accessible tools for simulating and training SNNs, understanding the theoretical underpinnings and low-level implementation is crucial—especially in scenarios where existing frameworks are incompatible with specialized hardware or minimal runtime environments.

The objective of this paper is to guide the reader from the theoretical foundations of spiking neurons to the practical implementation of a simple spiking neuron.

II. Biological and computational background

Biological sensory systems convert external stimuli — such as light, odors, touch, and taste — into discrete electrical events known as **spikes**. A spike is a voltage pulse that conveys information through the nervous system [5]. These signals are propagated along neural pathways to be processed, ultimately generating behavioral or physiological responses to the environment.

A biological neuron emits a spike only when the accumulated excitatory input at the soma exceeds a certain threshold. In the absence of such stimulation, the neuron remains inactive. This event-driven behavior makes biological neurons highly efficient in terms of energy consumption and computational resources.

Spiking Neural Networks (SNNs) aim to emulate some of these advantages by processing **spikes** at the input, hidden, and output layers. SNNs can also accept continuous-valued inputs retaining their event-driven properties and energy efficiency due to sparse activation patterns.

It is important to note that SNNs are **not** detailed simulations of biological neurons. Instead, they approximate specific computational characteristics inspired by neurobiology. More biologically faithful models — such as those explored in [4], which incorporate dendritic nonlinearities and other advanced neuronal features — have shown remarkable results in classification tasks, but at the cost of greater computational complexity.

As illustrated in Figure 1, SNN neurons exhibit strong noise tolerance when appropriately parameterized, functioning as **low-pass filters**. They can still generate reliable spikes under considerable interference. Moreover, SNN neurons are inherently time-sensitive, making them especially effective for processing temporal or sequential data streams [1].

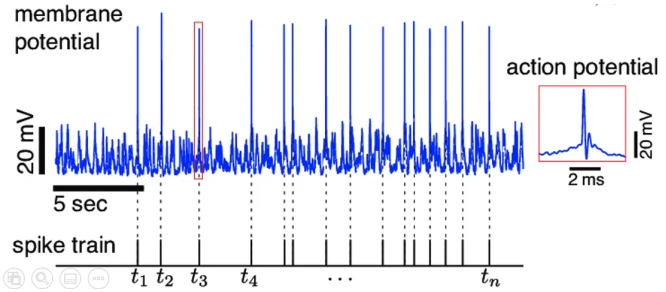

Fig. 1: Spikes from a noisy signal: The noisy signal in blue is filtered generating sparse spikes. Source [3]

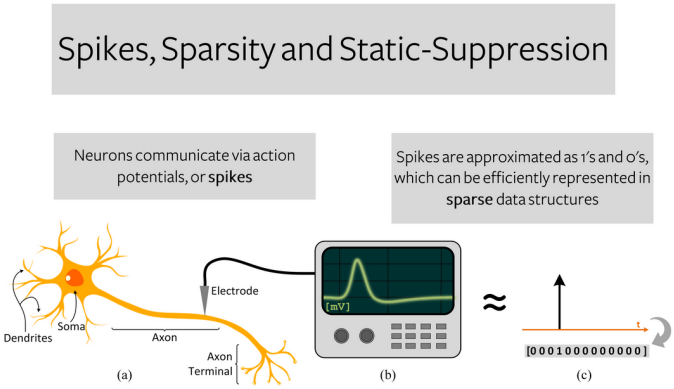

Fig. 3: Sparsity on Spike Neuron Networks. Source: [1]

A. *Leaky Integrate and Fire Neuron*

The focus of this work is the *Leaky Integrate-and-Fire* (LIF) neuron due to its simplicity, computational efficiency, and its strong generalization performance across many tasks [3].

There are more biologically accurate models, such as the **Hodgkin–Huxley neuron** [2], and the already cited in [4].

Unlike a standard artificial neuron, LIF integrates incoming weighted inputs over time with a *leakage* mechanism that continuously reduces the membrane potential, simulating the passive decay observed in biological membranes. Only when the membrane potential crosses a threshold, does the neuron emit a spike, after which the potential resets or decreases.

LIF behaves much like Resistor-Capacitor circuit as illustrated in Figure 2.

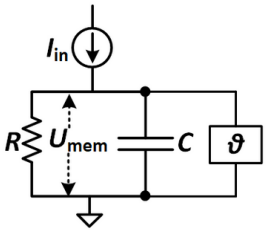

Fig. 2: RC model: R represents the leakage resistance, I_{in} the input current, C the membrane capacitance, U_{mem} the membrane potential (i.e., the accumulated charge) and ϑ a switch that lets the capacitor discharge (i.e. emit a spike) when a voltage threshold is reached. Source: [1]

Spikes are represented as **sparsely** distributed **ones** in a train of **zeros**, as illustrated in Figure 3 and Figure 4. This approach simplify the models and reduces the computational power and needed storage to run a SNN.

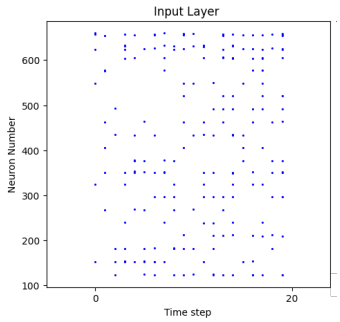

Fig. 4: Sparse activity of a SNN: The horizontal axis represents the time step of some data being processed the vertical are the SNN neuron number. Note that, most of the time, very few neurons gets activated. Source: The author.

As a result of the aforementioned characteristics, information in SNNs is encoded in terms of the *timing* and/or *rate* of spikes. This enables effective processing of temporal data streams but imposes limitations when dealing with static data like images which need some preprocessing in order to be processable by a spiking neural network.

These preprocessing steps may consist of repeated copies of the input data for temporal reinforcement, or of a progressive decomposition of the input information, enabling its presentation in temporally distributed segments.

B. *Mathematical Model of the LIF Neuron*

LIF model is governed by the Equations described in [1] bellow:

Let's consider Q as a measurement of electrical charge and $V_{mem}(t)$ is the membrane potential in a certain time t than the neuron capacitance C is given by Equation 1.

$$C = \frac{Q}{V_{mem}(t)} \tag{1}$$

Therefore the neuron charge can be expressed as Equation 2.

$$Q = C.V_{mem}(t) \tag{2}$$

To know how the charge changes according to the time (a.k.a. current) we can derivate Q as in Equation 3. This expression express the current in the capacitive part of the neuron I_C

$$I_C = \frac{dQ}{dt} = C.\frac{dV_{mem}(t)}{dt} \tag{3}$$

To calculate the total current passing by the resistive part of the circuit we may use the Ohm's law:

$$V_{mem}(t) = R.I_R \implies I_R = \frac{V_{mem}(t)}{R} \tag{4}$$

Than considering that the total current do not change, as seen in Figure 5, we have the total input current I_{in} of the neuron as in Equation 5.

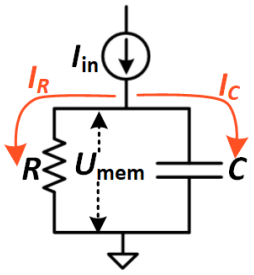

Fig. 5: RC model for currents: $I_{in} = I_R + I_C$

$$I_{in}(t) = I_R + I_C \implies I_{in}(t) = \frac{V_{mem}(t)}{R} + C.\frac{dV_{mem}(t)}{dt} \tag{5}$$

Therefore, to describe the passive membrane we got a linear Equation 6.

$$\begin{gathered} I_{in}(t) = \frac{V_{mem}(t)}{R} + C.\frac{dV_{mem}(t)}{dt} \implies \\ I_{in}(t) - \frac{V_{mem}(t)}{R} = C.\frac{dV_{mem}(t)}{dt} \implies \\ \boxed{R.I_{in}(t) - V_{mem}(t) = R.C.\frac{dV_{mem}(t)}{dt}} \end{gathered} \tag{6}$$

Than, if we consider $\tau = R.C$ as the **membrane time constant** we get voltages on both sides of Equation 7 which **describes the RC circuit**.

$$\begin{gathered} R.I_{in}(t) - V_{mem}(t) = R.C.\frac{dV_{mem}(t)}{dt} \implies \\ R.I_{in}(t) - V_{mem}(t) = \tau.\frac{dV_{mem}(t)}{dt} \implies \\ \boxed{\tau.\frac{dV_{mem}(t)}{dt} = R.I_{in}(t) - V_{mem}(t)} \end{gathered} \tag{7}$$

From that, and setting $I_{in} = 0$ (i.e. no input) and considering $\tau = R.C$ is a constant and a starting voltage $V_{mem}(0)$ the neuron's voltage behavior can be modeled as an exponential curve as can be seen in Equation 8.

$$\begin{gathered} \tau.\frac{dV_{mem}(t)}{dt} = R.I_{in}(t) - V_{mem}(t) \implies \\ \tau.\frac{dV_{mem}(t)}{dt} = -V_{mem}(t) = \\ e^{\ln(V_{mem}(t))} = e^{-\frac{t}{\tau}} = \\ \boxed{V_{mem}(t) = V_{mem}(0).e^{-\frac{t}{\tau}}} \end{gathered} \tag{8}$$

In summary: In the absence of an input I_{in}, the membrane potential decays exponentially as illustrated in Figure 6 and implemented in Listing 1.

```python
def lif(V_mem, dt=1, I_in=0, R=5, C=1):
  tau = R*C

  # e^(-dt/tau)
  decay_rate = np.exp(-dt/tau)

  # V_mem(t) = V_mem(0) * e^(-dt/tau)
  V_mem = V_mem + (V_mem + I_in*R) * decay_rate

  return V_mem
```

Listing 1: Python implementation of the action potential behavior of a LIF: **V_mem**: Membrane potential at the current time step; **dt=1**: simulation time step; **I_in=0**: No input current to the neuron; **R=5**: membrane resistance; **C=1**: membrane capacitance; **tau = R * C**: membrane time constant (i.e., the characteristic decay time of the membrane potential); **decay_rate=-dt/tau**: The rate by which the membrane voltage decays for each time step.

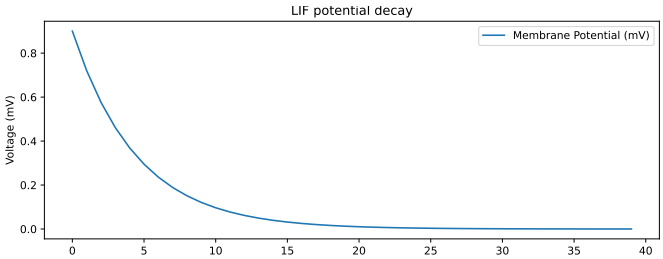

Fig. 6: Membrane potential decaying when $I_{in} = 0$: Time is represented by the horizontal axis. Neuron charge in Volts is the vertical axis. Source: The author

With the results from Equation 7 it is possible to calculate the action potential increasing as seen in Equation 9.

$$
\begin{aligned}
&\tau.\frac{dV_{mem}(t)}{dt} = R.I_{in}(t) - V_{mem}(t) = \\
&\frac{dV_{mem}(t)}{dt} + \frac{V_{mem}(t)}{\tau} = \frac{R.I_{in}(t)}{\tau} \implies \\
&\text{Integrating factor: } e^{\int \frac{1}{\tau} dt} = e^{\frac{1}{\tau}.t} \implies \\
&(e^{\frac{1}{\tau}.t}.V_{mem}(t))' = \frac{R.I_{in}(t)}{\tau}.e^{\frac{1}{\tau}.t} = \\
&\int (e^{\frac{1}{\tau}.t}.V_{mem}(t))' = \int \frac{R.I_{in}(t)}{\tau}.e^{\frac{1}{\tau}.t} dt = \\
&e^{\frac{1}{\tau}.t}.V_{mem}(t) = \int \frac{R.I_{in}(t)}{\tau}.e^{\frac{1}{\tau}.t} dt \therefore \\
&\text{Considering: } V_{mem}(t=0) = 0 \implies \\
&\boxed{V_{mem}(t) = I_{in}(t).R(1 - e^{\frac{1}{\tau}})}
\end{aligned}
\tag{9}
$$

Note that when action potentials increases there is still an exponential behavior as seen in Figure 7 and implemented in Listing 1.

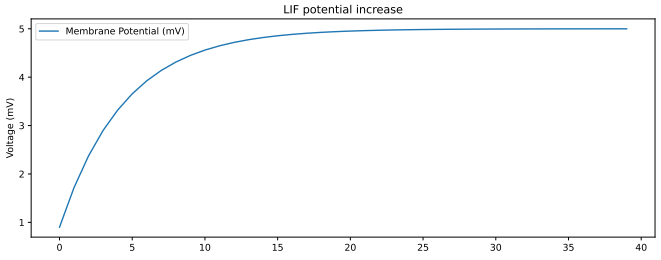

Fig. 7: Membrane potential increasing when setting I_in = 1 in Listing 1: Time is represented by the horizontal axis. Neuron charge in Volts is the vertical axis. Source: The author

Then taking into account some **threshold** which indicates a reset into the neuron potential and two type of resets (to zero and threshold subtraction) then it is possible to model the full LIF membrane charge and discharge behavior as depicted in Figure 8 and implemented in Listing 2:

```
def lif(V_mem, dt=1, I_in=0.5, R=5, C=1, V_thresh = 2, reset_zero = True):
  tau = R*C

  # e^(-dt/tau)
  decay_rate = np.exp(-dt/tau)

  # V_mem(t) = V_mem(0) * e^(-dt/tau)
  V_mem = V_mem + (V_mem + I_in*R) * decay_rate

  if V_mem > V_thresh:
    if reset_zero:
      V_mem = 0
    else:
      V_mem = V_mem - V_thresh
  return V_mem
```

Listing 2: Python implementation of the action potential full simulation of a LIF: **V_mem**: current membrane potential; **dt=1**: simulation time step; **I_in=0.5**: input current (assumed constant during dt); **R=5**: membrane resistance; **C=1**: membrane capacitance; **V_thresh=2**: firing threshold - when V_mem > V_thresh, the neuron "spikes"; **reset_zero=True**: determines if the neuron resets to zero after spiking; **tau = R * C**: membrane time constant (i.e., the characteristic decay time of the membrane potential); **decay_rate=-dt/tau**: The rate by which the membrane voltage decays for each time step.

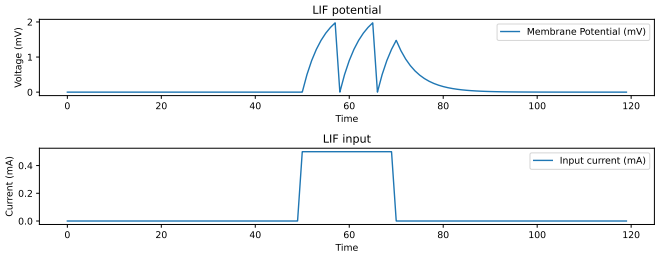

Fig. 8: Plot of the full simulated LIF: A 0.5 mA was provided from time 51 to 70.

C. Training

Until now just the *forward* pass has been shown, it's time to describe the backwards.

As seen, the SNN neuron has an activation-function behavior that is more relatable to a **step function**. Therefore, in principle, we can't use gradient descent-based solutions because this kind of function **is not** differentiable [5].

But there are some insights out there that may shed some light on this subject: While some *in vivo/in vitro* observations show that brains, in general, learn by strengthening/weakening and adding/removing synapses or even by creating new neurons or other cumbersome methods like RNA packets, there are some more straightforward ways like the ones listed bellow [5]:

- **surrogate gradient descent(chosen)**: Approximates the step function by using another mathematical function, which is differentiable (like a sigmoid), in order to train the network. These approximations are used only in the backward pass, while keeping the step function in the forward pass [5].
- **Spike Timing-Dependent Plasticity (STDP)**: If a pre-synaptic neuron fires **before** the post-synaptic one, there is a strengthening in connection, but if the post-synaptic neuron fires before, then there is a weakening.
- **evolving algorithms**: Use the selection of the fittest throughout many generations of networks.
- **reservoir/dynamic Computing**: **Echo state networks** or **Liquid state machines** respectively.

As stated, the **surrogate gradient descent** method will be used, and the chosen approximated activation function is

Hardtanh, which is defined in Equation 10, with its derivative given in Equation 11.

$$hardtanh(x) = \begin{cases} maxVal, & x < maxVal \\ minVal, & x > minVal \\ x, & otherwise \end{cases} \tag{10}$$

$$\frac{d}{dx} hardtanh(x) = \begin{cases} 0, & x < maxVal \\ 0, & x > minVal \\ 1, & otherwise \end{cases} \tag{11}$$

III. Implementation

Now that we known the inner workings of an spike neuron, it is time to implement the network.

Let's start by defining the architecture:

A. Architecture

This is a simple network: One layer responsible for the linear operations in which the weights belong and the spiking layer as depicted in Figure 9.

To facilitate coding the matrix operations need are made by eigen C++ library.

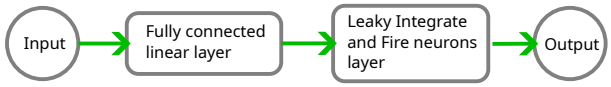

Fig. 9: Spiking neural network architecture

B. Linear layer

References

[1] Jason K. Eshraghian, Max Ward, Emre O. Neftci, Xinxin Wang, Gregor Lenz, Girish Dwivedi, Mohammed Bennamoun, Doo Seok Jeong, and Wei D. Lu. Training spiking neural networks using lessons from deep learning. *Proceedings of the IEEE*, 111(9):1016–1054, 2023.

[2] W. Gerstner, W.M. Kistler, R. Naud, and L. Paninski. *Neuronal Dynamics: From Single Neurons to Networks and Models of Cognition*. Cambridge University Press, 2014.

[3] Dan Goodman, Tomas Fiers, Richard Gao, Marcus Ghosh, and Nicolas Perez. Spiking Neural Network Models in Neuroscience - Cosyne Tutorial 2022, September 2022.

[4] Ilenna Simone Jones and Konrad Paul Kording. Can single neurons solve mnist? the computational power of biological dendritic trees, 2020.

[5] Nikola K Kasabov. *Time-space, spiking neural networks and brain-inspired artificial intelligence*. Springer, 2019.

IV. Appendix

A. Another interpretation of LIF

Starting with the Equation 7 and using Forward Euler Method to solve the LIF model:

$$\tau . \frac{dV_{mem}(t)}{dt} = R.I_{in}(t) - V_{mem}(t) \tag{12}$$

Solving the derivative in Equation 13 results in the membrane potential for any time $t + \Delta t$ in future:

$$\tau . \frac{V_{mem}(t + \Delta t) - V_{mem}(t)}{\Delta t} = R.I_{in}(t) - V_{mem}(t) =$$

$$V_{mem}(t + \Delta t) - V_{mem}(t) = \frac{\Delta t}{\tau} . (R.I_{in}(t) - V_{mem}(t)) =$$

$$\boxed{V_{mem}(t + \Delta t) = V_{mem}(t) + \frac{\Delta t}{\tau} . (R.I_{in}(t) - V_{mem}(t))} \tag{13}$$

That's all folks!!